

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I sanjay kumar soy(192311151) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

sanjay kumar soy

Signature of the student:

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption,

and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

Answer

1. Big Data Platform – Reverse Engineering the Architecture

A platform processing billions of website clicks, mobile interactions, and customer transactions requires a distributed architecture for high-volume ingestion, storage, processing, and analytics.

1. Distributed Storage

The probable storage system is HDFS or a cloud-based Data Lake such as Amazon S3.

- Stores huge amounts of data across multiple machines.
- Data is divided into blocks and distributed across nodes.
- Replication provides fault tolerance.
- Historical data can be retained for future analysis.

2. Data Ingestion

Apache Kafka is a likely ingestion framework for collecting continuous website, mobile, and transaction events.

Website / Mobile / Transactions

↓

Apache Kafka

↓

Storage + Processing

Kafka uses multiple partitions so that different consumers can process data simultaneously.

3. Batch Processing

For processing historical data, the platform can use:

- Apache Spark – fast distributed batch processing
- Hadoop MapReduce – large-scale batch processing
- Apache Hive – SQL-based analysis of large datasets

For example, daily customer transactions can be processed using Spark to identify sales and customer trends.

4. Structured and Semi-Structured Data

Structured data includes customer transactions, sales records, and product information. It can be stored in tables or data warehouses and queried using SQL.

Semi-structured data includes website logs, clickstreams, and JSON/XML data. It can be stored in a data lake and processed using Spark.

Formats such as Parquet and ORC can improve storage efficiency and query performance. Data can also be indexed or partitioned by fields such as date, customer ID, or region.

5. Stream Processing and Real-Time Analytics

For real-time processing, the platform could use:

- Apache Flink
- Spark Streaming
- Kafka Streams

Example:

User Click



Kafka



Flink / Spark Streaming



Real-Time Analytics



Dashboard / Alert

This allows the organization to monitor website activity, customer behavior, transactions, and potential fraud with low latency.

6. Scalability and Partitioning

The platform would use horizontal scaling, where additional servers are added when data volume increases.

Data can be partitioned by:

- Time/date – daily or monthly
- Customer ID
- Geographical region
- Product category

Kafka partitions allow parallel consumption, while HDFS distributes data blocks across nodes. Replication ensures that failure of one node does not cause data loss.

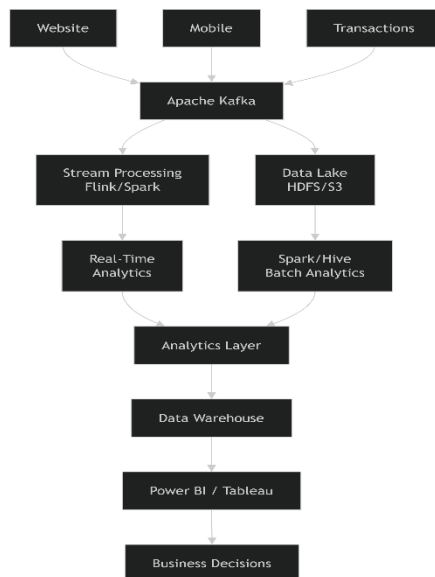
7. BI and Dashboard Integration

After processing, the data can be stored in a data warehouse or analytical database. Business intelligence tools such as Power BI, Tableau, or Looker can connect to this layer.

They can display:

- Sales and revenue
- Website traffic
- Customer behavior
- Conversion rates
- Product performance
- Real-time business metrics

Overall Architecture



2.E-Commerce Personalization Platform

An e-commerce platform generates large volumes of product views, cart additions, abandoned checkouts, payments, and orders. A suitable architecture would combine event streaming, distributed storage, real-time processing, caching, and machine learning.

1. Event Streaming and Data Collection

Apache Kafka is a likely event-streaming platform for collecting:

- Product views

- Cart additions/removals
- Checkout events
- Payment events
- User clicks

Kafka topics can separate events by type or region and allow multiple systems to consume the same data.

2. Customer Profiling and Synchronization

A customer profile database such as Redis, Cassandra, or MongoDB can maintain frequently changing customer information.

Session data, clickstream logs, and transactions can be synchronized using a common Customer ID/Session ID.

User Activity



Kafka



Stream Processing



Customer Profile



Recommendation Engine



Personalized Products

For example, if a customer views several laptops and adds one to the cart, the profile is updated immediately and relevant laptop accessories can be recommended.

3. Caching and Real-Time Personalization

Redis can be used as a high-speed caching layer to store:

- Active sessions
- Customer preferences
- Recently viewed products

- Current cart information
- Frequently requested recommendations

Caching reduces database access and provides low-latency recommendations.

4. Recommendation and Machine Learning

The recommendation workflow can use:

- **Collaborative Filtering** – recommends products based on similar customers.
- **Content-Based Filtering** – recommends products similar to those already viewed or purchased.
- **Hybrid Recommendation** – combines both approaches.

A typical ML pipeline is:

Historical Data



Data Cleaning



Feature Engineering



Model Training



Recommendation Model



Real-Time Prediction

5. Structured and Semi-Structured Data

Structured data includes:

- Orders
- Payments
- Product details
- Customer records

These can be stored in relational databases or data warehouses.

Semi-structured data includes:

- Clickstream events
- Session logs
- JSON browsing records

These can be stored in a **Data Lake/HDFS/S3**.

They can later be combined using a common **Customer ID, Product ID, and timestamp** for complete customer analytics.

6. Distributed Query and Analytics

Apache Spark SQL, Presto/Trino, or Hive can query large amounts of combined order and browsing data.

For example:

Product views + cart additions + completed orders → identify products with high views but low conversion rates.

7. Scalability and Fault Tolerance

The platform would use **horizontal scaling** by adding more processing nodes as traffic increases.

- Kafka partitions → parallel event processing
- Data replication → protects against node failures
- Distributed databases → handle large customer volumes
- Multiple processing nodes → prevent a single point of failure
- Cloud auto-scaling → increases resources during peak shopping periods

8. Dashboards and KPI Monitoring

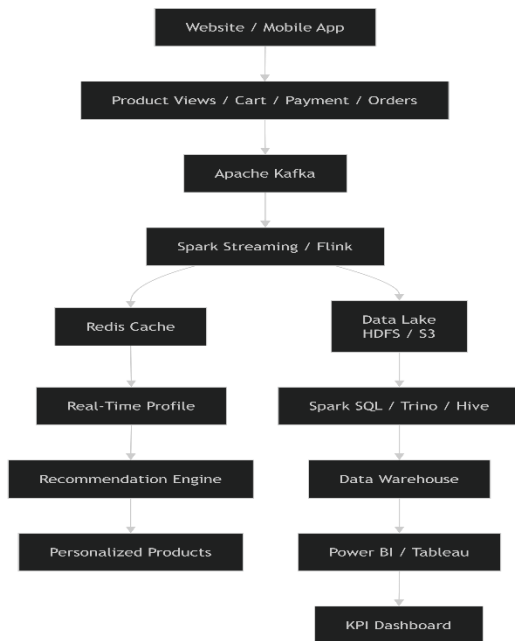
Processed data can be sent to a **data warehouse** and connected to **Power BI, Tableau, or Looker**.

Dashboards can monitor:

- Conversion rate
- Cart abandonment rate
- Sales and revenue
- Average order value
- Product views

- Customer engagement
- Recommendation click-through rate

Overall Architecture



3. Healthcare Big Data System

A healthcare platform handles different types of data such as patient histories, diagnostic images, wearable sensor data, and prescriptions. Therefore, it would likely use a hybrid architecture combining relational databases, NoSQL databases, and distributed storage.

Hybrid Database Architecture

Structured data such as patient records, prescriptions, billing, and laboratory results can be stored in MySQL/PostgreSQL or other relational databases.

Unstructured/semi-structured data such as medical images, sensor logs, and clinical documents can be stored in a data lake/HDFS or cloud object storage.

A NoSQL database such as MongoDB or Cassandra can manage high-volume wearable and clinical event data.

Patient Records —————→ Relational DB

Medical Images —————→ Data Lake / Object Storage

Wearable Streams —————→ NoSQL Database

Clinical Documents —————→ Data Lake

Data Integration

Hospitals, laboratories, pharmacies, and insurance systems may use APIs and ETL/ELT tools to exchange data.

Likely technologies include:

- **Apache Kafka** – real-time data exchange
- **Apache NiFi** – data flow and integration
- **ETL tools** – cleaning and transforming data
- **FHIR/HL7** – healthcare data exchange standards

A common Patient ID can be used to link records from hospitals, laboratories, and insurance providers.

3. Streaming and Machine Learning

Wearable devices continuously generate heart rate, blood pressure, glucose, and activity data.

Kafka + Apache Flink/Spark Streaming can process these streams in near real time.

Machine learning frameworks such as Python/scikit-learn, TensorFlow, or PyTorch can support:

- Disease prediction
- Patient risk prediction
- Abnormal vital-sign detection
- Readmission prediction
- Medical image classification

For example:

Abnormal heart rate → Kafka → Stream processing → ML model → Risk alert → Doctor dashboard.

4. Privacy, Encryption and Compliance

Healthcare data is highly sensitive, so the platform should implement:

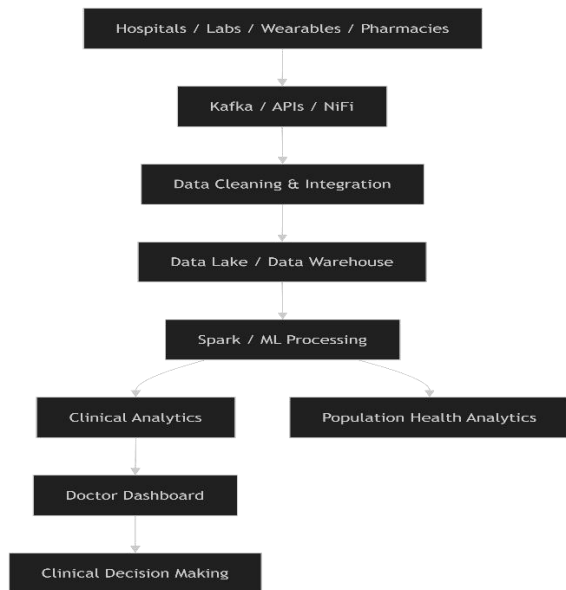
- **Encryption at rest and in transit**
- Role-Based Access Control (RBAC)
- Multi-factor authentication
- Data anonymization/pseudonymization
- Audit logs
- Secure APIs

- Data access monitoring
- Data retention policies

The system should comply with applicable healthcare and privacy regulations, such as HIPAA where applicable, along with relevant local data-protection requirements.

5. Analytics Pipeline

The analytics pipeline could be:



Historical data can be used for population health studies, while real-time data can provide alerts for individual patients.

6. Benefits

- Provides a unified view of patient information
- Enables real-time patient monitoring
- Supports early disease detection
- Helps doctors make data-driven decisions
- Enables large-scale population health research
- Maintains security and privacy of sensitive healthcare data

4.Smart City Big Data Platform

A smart city platform continuously generates data from **traffic sensors, CCTV systems, weather stations, and public transport systems**. A distributed architecture is required to process this data in real time while also storing historical information for urban planning.

1. Edge Computing Setup

Edge computing can process data close to where it is generated.

Traffic Sensors / CCTV / Weather / Buses



Edge Devices



Local Filtering & Analysis



Central Data Platform

Edge devices can filter unnecessary CCTV metadata, detect traffic conditions, and perform initial processing. This **reduces network bandwidth and latency**.

2. Distributed Messaging and Storage

Apache Kafka is a probable messaging system for continuously collecting sensor events.

- Kafka handles high-volume real-time streams.
- Topics can separate traffic, weather, CCTV, and transport data.
- Partitions allow parallel processing.

Centralized storage could use:

- **HDFS / Cloud Data Lake** for large historical datasets.
- **Object storage such as Amazon S3** for scalable storage.
- Data can be replicated for fault tolerance.

3. Geospatial and Stream Analytics

Traffic data contains location information such as **GPS coordinates, road IDs, and geographic regions**.

Apache Spark Streaming or Apache Flink can process traffic streams in real time.

Geospatial analytics can identify:

- Congested roads
- Traffic hotspots
- Accident-prone areas
- Public transport delays
- Alternative routes

Machine learning models can use historical traffic, weather, time, and location data to **predict future congestion**.

4. Databases for Time-Series and Location Data

A **time-series database** such as InfluxDB or TimescaleDB can store continuously changing sensor measurements.

For location-aware queries, PostgreSQL with PostGIS is suitable because it supports geographic objects and spatial queries.

For example:

Find all traffic sensors within 5 km of a congested road.

5. Batch + Real-Time Analytics

The platform can combine both approaches:

Real-time analytics:

- Detect current congestion
- Monitor buses
- Generate immediate alerts
- Monitor weather conditions

Batch analytics:

- Analyze months/years of traffic data
- Identify long-term congestion patterns
- Plan new roads and public transport routes
- Study population and transportation trends

6. Security and Access Control

Citizen and infrastructure data should be protected using:

- **Encryption** during transmission and storage
- Role-Based Access Control (RBAC)
- Authentication and multi-factor authentication
- Secure APIs
- Network segmentation

- Audit logs
- Data anonymization for sensitive citizen information
- Regular security monitoring

For example, traffic operators may access traffic data, while only authorized administrators can access sensitive infrastructure information.

5. Healthcare Analytics System

A healthcare analytics platform integrates patient records, wearable readings, laboratory reports, and appointment histories. Since the data is both sensitive and available in different formats, a hybrid big data architecture is required.

1. Hybrid Storage Architecture

The platform would likely use different storage systems for different types of data:

- **Relational database (PostgreSQL/MySQL):** Patient records, appointments, prescriptions, and billing data.
- **NoSQL database (MongoDB/Cassandra):** High-volume wearable sensor readings and semi-structured clinical data.
- **Data Lake (HDFS/Cloud Object Storage):** Medical documents, lab reports, images, and historical raw data.
- **Data Warehouse:** Cleaned and integrated data for analytics and reporting.

Patient Records —————→ Relational DB

Wearable Data —————→ NoSQL Database

Lab Reports/Images ———→ Data Lake

Historical Data —————→ Data Warehouse

2. ETL and Interoperability

ETL/ELT pipelines can extract data from hospitals and clinics, transform it into common formats, and load it into the analytics platform.

Probable technologies:

- **Apache NiFi** – data flow and integration
- **Apache Kafka** – real-time data ingestion
- ETL tools for cleaning and transformation
- **HL7 and FHIR** – healthcare interoperability standards

A common **Patient ID** can be used to connect records from different hospitals, clinics, laboratories, and wearable devices.

3. Real-Time Monitoring

Wearable devices continuously generate readings such as:

- Heart rate
- Blood pressure
- Blood glucose
- Oxygen level

Kafka + Apache Flink/Spark Streaming can process these readings in real time.

Wearable Device



Kafka



Flink / Spark Streaming



Health Risk Detection



Doctor / Patient Alert

For example, an abnormal heart-rate pattern can trigger an immediate alert.

4. Predictive Health Analytics

Machine learning models can analyze historical and real-time patient data for:

- Disease risk prediction
- Readmission prediction
- Patient deterioration
- Diagnosis risk scoring
- Treatment outcome prediction

Models such as Random Forest, Logistic Regression, XGBoost, and neural networks can be used depending on the problem.

5. Distributed Processing for Population Analysis

Frameworks such as Apache Spark and Hadoop can process millions of patient records across multiple machines.

They can identify:

- Disease trends across populations
- Risk factors
- Regional health patterns
- Treatment outcomes
- Hospital utilization trends

Distributed processing provides parallel execution, scalability, and faster analysis compared with a single centralized system.

6. Security, Compliance and Auditing

Healthcare data requires strict protection.

The platform would likely implement:

- **Encryption at rest and in transit**
- Role-Based Access Control (RBAC)
- Multi-factor authentication
- Data masking and anonymization
- Secure APIs
- Detailed **audit logs**
- Data access monitoring
- Backup and disaster recovery

The organization should comply with applicable privacy and healthcare regulations, such as HIPAA where applicable, along with relevant local data-protection laws.

7. Machine Learning for Diagnosis and Treatment

ML models can generate a **risk score** based on patient history, lab results, symptoms, and wearable readings.

Patient History + Lab Results

+

Wearable Data

↓

Feature Engineering

↓

ML Model

↓

Risk Score

↓

Doctor's Clinical Decision

Recommendation systems can help identify potentially suitable treatments based on historical outcomes and patient characteristics, but such outputs should support—not replace—qualified clinical judgment.

The probable architecture combines relational databases, NoSQL storage, data lakes, Kafka, NiFi, FHIR/HL7, Spark/Flink, and machine learning. Real-time streaming enables continuous patient monitoring, while distributed processing supports large-scale population health analysis. **Encryption, access control, compliance monitoring, and audit logs** protect sensitive medical information, while ML models provide useful risk scoring and decision-support insights.